Problem A. Wowoear

Let the original polyline length be S. If the shortcut endpoints are a and b, and the original-route distance from a to b is $P(a, b)$, then the final route length is

$$S - P(a, b) + |a - b|.$$

So we maximize the saving $P(a, b) - |a - b|$ among all visible shortcut segments.

The main observation is a perturbation argument. In an optimum, some constraint is tight: either an endpoint is a vertex of the original polyline, or the supporting line of the shortcut is blocked by a vertex. Otherwise the shortcut can be moved or rotated slightly while the same edges remain visible, improving the objective until a vertex becomes critical.

Enumerate the critical vertex p. Sort the directions from p to all other vertices by polar angle. Between two adjacent directions, the first original segment hit by a ray from p on each side is unchanged. Thus, inside one angular interval, the two touched edges are fixed and the shortcut value is a one-variable convex/unimodal function of the angle. Use ternary search for the interior-contact case, and check boundary directions for endpoint cases.

For every generated candidate, explicitly verify that the open shortcut segment intersects no polyline segment except at its endpoints. The answer is the original length minus the best saving. A straightforward implementation is fast enough for $n \le 200$; the numeric search contributes the usual logarithmic precision factor.

Problem B. Mine Sweeper II

For a mine-sweeper map, the sum of all numbers on non-mine cells counts adjacent pairs consisting of one mine cell and one non-mine cell. Complementing every cell preserves this count, because each mixed adjacent pair remains mixed.

Therefore the score of A and the score of the complement $\overline{A}$ are both equal to the required score. We only need to transform B into either A or $\overline{A}$. The two Hamming distances add up to nm, so at least one of them is at most $\lfloor nm/2 \rfloor$. Output the closer one.

The time complexity is $\mathcal{O}(nm)$.

Problem C. Sum of Log

Because $i \,\&\, j = 0$, the addition $i + j$ has no carries, so $i + j = i \oplus j$. Hence

$$\lfloor \log_2(i + j) \rfloor + 1$$

is the position of the highest set bit of $i \oplus j$, plus one. The pair $(0, 0)$ contributes 0.

Use digit DP from high bits to low bits. The state stores the current bit, the highest bit already set in either number, and two tight flags for the bounds X, Y. At each bit, try only $(0, 0), (1, 0), (0, 1)$; the choice $(1, 1)$ is forbidden. When a 1 appears before any higher bit has appeared, record this bit as the highest set bit.

At the end of the DP, add $h + 1$ if a highest bit h was recorded. There are only about $30 \cdot 30 \cdot 2 \cdot 2$ states, so each test case is handled in $\mathcal{O}(\log X \log Y)$ time.

Problem D. Walker

For one walker starting at p with speed v, the minimum time needed to cover an interval $[l, r]$ containing p is

$$\frac{(r - l) + \min(p - l, r - p)}{v}.$$

The walker first goes to the nearer endpoint of the interval and then to the other endpoint.

Binary search the answer T. For a fixed T, compute for each walker the farthest prefix $[0, x]$ he can cover and the farthest suffix $[x, n]$ he can cover using the formula above. The segment can be fully covered if one walker can cover a prefix and the other can cover an overlapping suffix, in either assignment.

The predicate is monotone in T, so binary search gives the optimum. The complexity per test case is $\mathcal{O}(\log(1/\varepsilon))$ with closed-form reach computation, or with an extra tiny constant if the reach values are also found by binary search.

Problem E. The Journey of Geor Autumn

Let f_i be the number of good permutations of length i. The minimum value 1 must appear in one of the first k positions; otherwise it violates the condition when it appears.

Suppose 1 is placed at position j ($1 \le j \le k$). The $j-1$ positions before it can be chosen and ordered freely, and the remaining suffix becomes the same problem of size $i-j$. Therefore

$$f_i = \sum_{j=1}^{k} \binom{i-1}{j-1} (j-1)! f_{i-j}.$$

Define $g_i = f_i/i!$. Dividing by $i!$ gives

$$g_i = \frac{1}{i} \sum_{j=1}^{k} g_{i-j}.$$

Maintain prefix sums of g and compute each transition in $\mathcal{O}(1)$. Finally output $g_n n!$ modulo 998244353.

The time complexity is $\mathcal{O}(n)$.

Problem F. Fountains

For an interval $[l, r]$, write

$$W(l, r) = \sum_{i=l}^{r} s_i.$$

If a set of plans P is chosen, the saved value for an announced interval $[L, R]$ is

$$\max_{[l,r] \in P,\ L \le l \le r \le R} W(l, r),$$

where the maximum is 0 if no chosen plan is contained in $[L, R]$. Thus we maximize the total saved value for each number of chosen plans, and subtract it from the total loss $\sum_{L \le R} W(L, R)$.

Sort all candidate intervals in decreasing order of $W(l, r)$. When the current interval $[l, r]$ is selected, it becomes the best plan exactly for those outer intervals that contain $[l, r]$ and were not already fixed by a previously selected interval of larger value.

Represent the fixed outer intervals by thresholds t_L. For each left endpoint L, let t_L be the smallest right endpoint R such that some already selected interval $[l', r']$ with $L \le l' \le r' \le R$ is contained in $[L, R]$; set $t_L = n + 1$ if none exists. Then $[L, R]$ is fixed iff $R \ge t_L$. Selecting $[l, r]$ changes

$$t_L \leftarrow \min(t_L, r) \qquad (1 \le L \le l),$$

and newly fixes $\sum_{L=1}^{l} \max(0, t_L - r)$ outer intervals, each gaining value $W(l, r)$.

Now do DP over the sorted intervals, the number of chosen plans, and the monotone vector $(t_1, \ldots, t_n)$. Since $n \le 9$, the number of possible vectors is small. Let $best_k$ be the maximum total saved value after choosing k intervals; output

$$\sum_{1 \le L \le R \le n} W(L, R) - best_k$$

for all k.

Problem G. Fibonacci

The Fibonacci sequence modulo 2 is periodic:

$$1, 1, 0, 1, 1, 0, \ldots$$

Thus f_i is even iff $3 \mid i$. Let

$$e = \left\lfloor \frac{n}{3} \right\rfloor, \qquad o = n - e.$$

A pair contributes 1 iff at least one of the two Fibonacci numbers is even. Therefore the answer is

$$\binom{n}{2} - \binom{o}{2} = e \cdot o + \binom{e}{2}.$$

This is computed in $\mathcal{O}(1)$ time.

Problem H. Rice Arrangement

There is an optimal assignment in which the segments connecting guests to assigned bowls do not cross in cyclic order. If two assignment edges cross, swapping the two bowls does not increase the necessary rotation interval and reduces the number of crossings.

After sorting guests and bowls on the circle, enumerate which bowl is matched to the first guest; the rest of the matching is then determined cyclically. Try both clockwise and counterclockwise orders.

For one fixed matching, lift positions to integers. If guest i is at a_i and the assigned bowl is at b_i, the table must visit the rotation offset

$$d_i = a_i - b_i.$$

Let $l = \min_i d_i$ and $r = \max_i d_i$. Starting from offset 0, the shortest walk on a line that visits all offsets in $[l, r]$ has length

$$(r - l) + \min(-l, r).$$

Taking the minimum over all cyclic shifts and directions solves the problem in $\mathcal{O}(k^2)$ time.

Problem I. Sky Garden

The intersection points are described by a radius $r \in \{1, \ldots, n\}$ and an angular index $t \in \{0, \ldots, 2m-1\}$. The angle step is $\theta = \pi/m$. If $m > 1$, the center is also a point because it is the intersection of two straight roads.

For two points with radii $r_1 \le r_2$ and angular difference $d \in [0, m]$, the shortest path distance is

$$(r_2 - r_1) + \min(r_1 d\theta, 2r_1).$$

The first term moves radially; the second term either follows the smaller circle or goes through its center.

For a fixed radius r, precompute

$$S(r) = 2 \sum_{d=1}^{m-1} \min(rd\theta, 2r) + 2r,$$

where the last term is the opposite direction $d = m$. For equal radii, the unordered angular-pair contribution is $mS(r)$. For $r_1 < r_2$, the contribution over all angular positions is

$$2m\big(2m(r_2 - r_1) + S(r_1)\big).$$

If $m > 1$, also add the center-to-circle distances $\sum_{r=1}^{n} 2mr$.

The total complexity is $\mathcal{O}(nm + n^2)$.

Problem J. Octasection

Sweep the plane $x = a$ from $-\infty$ to ∞. A cuboid intersected by this plane is already touched. Every other cuboid must be touched by $y = b$ or by $z = c$, so in the yz-plane the point (b, c) must lie in the cross

$$[y_{min}, y_{max}] \times \mathbb{R} \quad \cup \quad \mathbb{R} \times [z_{min}, z_{max}].$$

Thus, for the current sweep position, we must decide whether the intersection of many crosses is nonempty.

The complement of one cross is the union of four open corner rectangles. Taking the union of all upper-left forbidden corners forms a monotone contour; the other three corners form three analogous contours. For each compressed y coordinate, the feasible z values are between the maximum of two lower contours and the minimum of two upper contours.

Maintain these four contours during the sweep. The y coordinates are split into four classes according to which lower contour and which upper contour are active. For each class, a segment tree stores

$$\text{upper}(y) - \text{lower}(y)$$

and can answer whether some nonnegative value exists. If it exists, recover the corresponding y, z and output the current a, b, c.

Contour endpoints appear or disappear only a constant number of times: when the cuboid starts/stops being crossed by the x-plane, and when the endpoints that block it disappear. Processing these events with balanced sets and segment trees gives an $\mathcal{O}(n \log n)$ implementation.

Problem K. Traveling Merchant

Build the graph H consisting only of edges whose endpoints have different initial colors. A legal path using only new vertices must use edges of H. Once the merchant can reach a previously visited vertex, he can repeat the path after that previous visit forever. In the initial graph, this means an alternating path whose final vertex has an edge to an earlier vertex of the same color.

Therefore, for every same-color edge (u, v), check whether H contains a simple path

$$0 \rightsquigarrow u \rightsquigarrow v \quad \text{or} \quad 0 \rightsquigarrow v \rightsquigarrow u.$$

If yes, the same-color edge closes the repeatable cycle and the answer is `yes`.

Use Tarjan's algorithm on H to decompose it into vertex-biconnected components, and build the block-cut tree. For a same-color edge (u, v), let w be the LCA of u and v in a DFS tree of the component reachable from 0. The required simple path exists exactly when u or v is in the same vertex-biconnected component as w. This becomes a subtree-containment check on the block-cut tree.

The algorithm is $\mathcal{O}(n + m)$ with linear LCA preprocessing, or $\mathcal{O}((n + m) \log n)$ with binary lifting.

Problem L. Traveling in the Grid World

A walk from (x_1, y_1) to (x_2, y_2) is allowed iff

$$\gcd(|x_1 - x_2|, |y_1 - y_2|) = 1,$$

because otherwise another grid point lies on the segment.

If $\gcd(n, m) = 1$, the direct segment is legal and the answer is

$$\sqrt{n^2 + m^2}.$$

Otherwise, two segments are sufficient. Choose an intermediate point (p, q) such that

$$\gcd(p, q) = 1, \qquad \gcd(n - p, m - q) = 1,$$

and the two directions are not the same. The length is

$$\sqrt{p^2 + q^2} + \sqrt{(n - p)^2 + (m - q)^2}.$$

By convexity, the optimum lies near the straight line from $(0, 0)$ to (n, m). Enumerate one coordinate and test only a constant-size neighborhood of the nearest integer point on the line, for example

$$q \approx \frac{pm}{n}$$

when p is fixed, and symmetrically for $p \approx qn/m$. Check the gcd and direction constraints and keep the best candidate.

The complexity is $\mathcal{O}(\max(n, m))$ per test case.

Problem M. Gitignore

Build a trie of all paths. Mark leaves corresponding to files that must be kept. For each node, determine whether its subtree contains any kept file.

If a non-root node contains no kept file, the entire subtree can be ignored by one gitignore line, so the DP value is 1. Otherwise the node cannot be ignored as a whole, and the value is the sum of the values of its children. A kept-file leaf contributes 0; an ignored-file leaf contributes 1.

The root is special because an empty gitignore line cannot ignore the whole project, so the root always uses the sum of its children. The time complexity is linear in the total input path length.